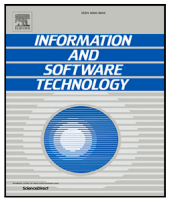
Information and Software Technology 170 (2024) 107445

Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof



Source code expert identification: Models and application

Otávio Cury [a,*], Guilherme Avelino [a], Pedro Santos Neto [a], Marco Túlio Valente [b], Ricardo Britto [c,d]

[a] Federal University of Piauí, Teresina, Piauí, Brazil
[b] Federal University of Minas Gerais, Belo Horizonte, Minas Gerais, Brazil
[c] Blekinge Institute of Technology, Karlskrona, Blekinge, Sweden
[d] Ericsson, Karlskrona, Blekinge, Sweden

ARTICLE INFO

Keywords:
Source code expertise
Machine learning
Truck Factor

ABSTRACT

Context: Identifying source code expertise is useful in several situations. Activities like bug fixing and helping newcomers are best performed by knowledgeable developers. Some studies have proposed repository-mining techniques to identify source code experts. However, there is a gap in understanding which variables are most related to code knowledge and how they can be used for identifying expertise.
Objective: This study explores models of expertise identification and how these models can be used to improve a Truck Factor algorithm.
Methods: First, we built an oracle with the knowledge of developers from software projects. Then, we use this oracle to analyze the correlation between measures from the development history and source code knowledge. We investigate the use of linear and machine-learning models to identify file experts. Finally, we use the proposed models to improve a Truck Factor algorithm and analyze their performance using data from public and private repositories.
Results: *First Authorship* and *Recency of Modification* have the highest positive and negative correlations with source code knowledge, respectively. Machine learning classifiers outperformed the linear techniques (*F-Score* = 71% to 73%) in the largest analyzed dataset, but this advantage is unclear in the smallest one. The Truck Factor algorithm using the proposed models could handle developers missed by the previous expertise model with the best average *F-Score* of 74%. It was perceived as more accurate in computing the Truck Factor of an industrial project.
Conclusion: If we analyze *F-Score*, the studied models have similar performance. However, machine learning classifiers get higher *Precision* while linear models obtained the highest *Recall*. Therefore, choosing the best technique depends on the user's tolerance to false positives and negatives. Additionally, the proposed models significantly improved the accuracy of a Truck Factor algorithm, affirming their effectiveness in precisely identifying the key developers within software projects.

1. Introduction

Software evolution is an iterative process mainly based on repeated source code changes in various development-related activities [1]. These activities require seamless collaboration and efficient management of the development team. However, this management becomes particularly complicated in large and geographically distributed projects, where project managers need as much information as possible about their development to coordinate their teams effectively. In this context, it is essential to know who has the expertise in which parts of the source code, especially in a context where remote work has grown fast and face-to-face interactions have been reduced [2].

Identifying developer expertise is valuable in many scenarios in software development. For example, it can be used in task assignments (e.g., who can assist a newcomer, and who is suitable for fixing a bug) [3]. Furthermore, this information can help project managers monitor knowledge concentration, where a few team members possess vital knowledge about the code, a situation that poses risks of discontinuation to the project [4,5]. However, keeping track of project file familiarity is challenging for developers and managers due to the amount of information related to changes that they handle [6]. To help with this task, we can rely on information available in Version Control Systems (VCS), wherein a large part of the developer-file iterations is logged. Using such information, previous research has been conducted

* Corresponding author.
E-mail addresses: otaviocury@ufpi.edu.br (O. Cury), gaa@ufpi.edu.br (G. Avelino), pasn@ufpi.edu.br (P.S. Neto), mtov@dcc.ufmg.br (M.T. Valente), rbr@bth.se (R. Britto).

https://doi.org/10.1016/j.infsof.2024.107445
Received 8 May 2023; Received in revised form 13 March 2024; Accepted 14 March 2024
Available online 16 March 2024

to address the file expert identification problem [6,7]. For example, in previous research, Avelino et al. compared the performance of three techniques for identifying file experts [8]. They identified opportunities for improving these techniques by exploring additional variables, such as *file size* and the *recency* of modifications.

Inspired by these findings, this paper investigates the correlation between twelve variables extracted from developmental history and the source code knowledge. Subsequently, we employ machine learning classifiers and linear models, assessing their efficacy in identifying experts in source code files across a substantial dataset encompassing both public and industrial systems. Furthermore, we scrutinize the practicality of deploying these expert identification models in the computation of the *Truck Factor* (or *Bus Factor*). This metric indicates how many developers need to leave the project, before the project is in serious trouble [9,10], helping to identify knowledge concentration in software projects. This metric is widely studied in the literature and perceived by the community as an essential metric of a project's health [4,5,9,11–13]. This study can be divided into two sub-studies, as follows:

In the first study, we study which variables present in *VCS* are more related to source code knowledge based on a dataset composed of the development history of 111 open-source and two industrial systems. We then propose and compare the performance of knowledge models in identifying experts in source code files. We seek to answer the following research questions:

- ***RQ1:*** *How do repository-based metrics correlate with developer's knowledge?*
  Motivation: Several works in the literature use different repository-based metrics to infer the knowledge of developers in source code files. However, we did not identify large-scale studies that correlated these variables with knowledge. By answering this question, we seek to understand how these variables are related to knowledge in source code, which can guide the development of models that estimate knowledge and identify source code experts.
- ***RQ2:*** *How do machine learning classifiers compare with traditional techniques for identifying source code experts?*
  Motivation: Due to the vastly successful application of machine learning classifiers in the software engineering literature, we also evaluated if the application of machine learning classifiers can improve the performance in identifying experts achieved by other techniques in previous works.

In the second study, we examine the viability of deploying these expert models to compute the *Truck Factor* of 130 open-source projects and one industrial project. We modify Avelino's Truck Factor algorithm [5] and assess whether, through the proposed models, it incorporates important developers with more recent contributions—an identified weakness in the original study involving open-source projects [5]. In addition to this quantitative analysis, we also perform a qualitative investigation of the Truck Factor computations in an industrial system. We surveyed with tech leaders of the project, comparing the *Truck Factor Developers* lists generated by the algorithm using these different knowledge models. This practical application seeks to answer the following research question:

- ***RQ3:*** *Can the proposed models improve a state-of-the-art Truck Factor algorithm?*
  Motivation: Avelino et al. proposed a greedy algorithm to calculate the Truck Factor of software repositories [5]. This algorithm was also used for similar purposes in recent studies [4,9,12–15]. However, in the original work, the authors pointed to the algorithm's deficiency in including experts with recent contributions to the projects in the Truck Factor Developers. With models that consider the recency of the contributions, we modify Avelino's algorithm to verify if it addresses the deficiency pointed out in the study.

The study we report here is an expansion of a conference paper [16], in which we compare the performance source code knowledge models in identifying file experts. In this paper, we extend the previous work in two major directions:

1. We present a new linear model named *Degree of Expertise* for calculating knowledge in code files. With this new model, we increase both the performance comparison of the studied techniques and the application of these techniques in the computation of Truck Factors.
2. We describe an application of the studied models in the improvement of a Truck Factor algorithm. We evaluate the performances of the modified algorithm in two scenarios. First, in the identification of important missing developers of open-source projects pointed out in the original study, and second, in the accuracy of the Truck Factor of a private project, comparing the different models in a survey with the project's development leaders.

The remainder of this paper is organized as follows. Section 2 presents related works. In Section 3, we present the investigation of the knowledge models, describing the procedure adopted to select the target systems, the evaluated models, and the results and discussions of their performances in identifying experts in source code files. In Section 4, we present the results of applying the evaluated models on a Truck Factor algorithm. Section 5 lists threats to the validity of both studies' results. Finally, Section 6 concludes by presenting our key findings.

2. Related work

This section covers the literature related to our research. Section 2.1 presents literature that proposes and compares techniques to identify developers' expertise on source code, and Section 2.2 includes studies that propose methods of Truck Factor computation.

2.1. Source code knowledge models

Some studies are based mainly on information about changes such as the number of commits and who made the last change to identify expertise. Hossen et al. [17] presented an approach that identifies experts associated with a change request based on who last changed the files. Others count the number of changes made on source code [18–20]. Other models, such as the one proposed by Sülün et al. use the number of commits in the artifact and related ones to recommend code reviewers [21]. However, based on previous work [8,22], we believe these variables alone are not enough, and in this study, we analyze more variables and their relationship with source code knowledge.

In addition to the number of changes, other studies also use the number of iterations a developer has with a file. For example, the *Degree of Knowledge* (DOK) model, proposed by Fritz et al. has a component of file's authorship, the *Degree of Authorship* (DOA), but also counts the number of interactions that the developer have with the file, the *Degree of Interest* (DOI) [6]. Although the authors demonstrated that interaction data can enhance expert identification, the computation of *DOI* requires plugins in the development environment, which complicates its usage in large studies. In terms of differences from the models studied in this work, *DOK* does not directly address recency and does not consider file size—two variables highlighted as factors in the calculation of knowledge [8,22].

Other techniques calculate the impact of time on the knowledge of source code (recency). Lucas et al. use a hyperbolic function that models forgetting and relearning, which, together with the number of iterations with the artifact, calculates a developer's knowledge [23]. Silva et al. presented a model that computes the developer's expertise in an artifact based on the number of changes made by a developer using time windows [7]. Other approaches that consider the recency

of changes appear in studies focused on the recommendation of developers for the resolution of change requests. Kagdi et al. [24] proposed an approach that locates source code files relevant to a given change request and identifies experts in those files prioritizing developers who made more commits [3,25]. Studies such as Tüzün and Dogrusöz [26] and Asthana [27] use information on modification recency, focusing on recommending code reviewers. For the same purpose, Chouchen [28] uses the recency of review comments to recommend the most suitable developers to review a code change. Still, they did not present an in-depth analysis showing how the variables were used to suit this identification.

Regarding the source used to extract knowledge information in source code, we focus only on data present in Version Control Systems (VCS). Some approaches use other sources such as the number of interactions with a file [23], code reviews [27], review comments [28], and numbers of meetings related to commits [9]. While these are useful sources, they depend on specific tools, such as plugins in the development environment and the use of company-specific tools. Due to the universality of *VCS*, its use as a data source becomes more practical.

Regarding the use of machine learning, Montandon [29] investigated the performance of supervised and unsupervised classifiers in identifying library experts. Even though we followed a similar data collection and analysis process, our work has a distinct goal. We rely on classifiers for identifying experts at source code files, while Montandon targets the identification of experts at libraries and frameworks. Other examples of machine learning applications in the context of developer expertise target the bug assignment problem [30], which is a different problem from the one investigated here. In summary, we have not identified studies that investigate the performance of machine learning in the classification of file experts based on *VCS* information.

Some authors compared expertise identification techniques. Avelino et al. compared the performance of *Commits*, *Blame*, and *DOA* in identifying source code file maintainers [8]. The results showed that all three techniques have similar performance and also pointed out the importance of considering the *recency of the modifications* and the *file size* as a strategy to improve these techniques. Hannebaur et al. [31] compared the performance of eight algorithms to recommend code reviewers. Of these algorithms, six are based on expertise by modification, and two are based on review expertise. Finally, Anvik and Murphy [32] compared two approaches to determine appropriate developers for resolving bug reports: one uses the *Line 10* to define which developers are experts in the files associated with the bug report, and the other uses data from bug networks. The work presented in this paper can be distinguished from these three works in three key aspects: purpose, compared techniques, and scope. In terms of research purpose, our work aligns with two studies that also compare expert identification techniques, Avelino et al. [8] and Anvik and Murphy [32]. However, we use to compare different techniques, particularly machine learning classifiers. Regarding the compared techniques, Avelino et al. [8], Krüger et al. [22], and Hannebauer et al. [31] used some of the baseline techniques selected by us. However, we also investigate the performance of machine learning models. Finally, regarding the scope of the studies, none of them used data from a similar number of repositories as in our study.

2.2. Truck factor computation

We can highlight three studies that investigated ways to estimate truck factors. First, Consentino et al. proposed that the Truck Factor of software can be found with the concepts of *primary and secondary developers* [10]. Primary developers (P) are the developers who have modified a minimum X percentage of the artifact, and the secondary developers (S) are the ones who have modified at least $Y < X$. The Truck Factor of the artifact is given $|P\ U\ S|$. Second, Rigby et al. use authorship of lines and compute the Truck Factor [33]. A line is *owned* by the developer who last modified it, and a file is considered abandoned when at least 90% of its lines are *unowned*. The authors analyze Truck Factor scenarios by removing developers in a Monte Carlo simulation. Finally, Avelino et al. use the *Degree of Authorship* to identify *authors* of files and calculate the Truck Factor [5]. The authors proposed an algorithm that simulates the abandonment of the project's core developers and identifies the impact with the number of files without *authors* (i.e., developers with high authorship in a file). The algorithm stops and returns the Truck Factor when more than 50% of the project's files missed their authors. These three mentioned approaches had their performances compared by a study by Ferreira et al. [4]. Their performances were compared on the truck factor computation and on the accuracy in identifying the right truck factor developers. The results show that Avelino's algorithm is the most accurate in these two aspects.

Other studies use the algorithm proposed by Avelino cited above [5]. Avelino et al. use it to investigate open-source projects that faced abandonment by core developers, how many survived, and how [13]. Jabrayilzade et al. use it as a baseline to compare the accuracy of a proposed algorithm derived from [9]. Celefato et al. use it to identify core developers of open-source software to investigate their downtime and disengagement of projects [14]. Canedo et al. to gain insights into segregation issues of women in *OSS* communities [15]. Almarimi et al. use it to detect community smells in software projects [12]. Finally, Karlsson studies this algorithm and variations of it in a study to analyze and compare Truck Factors from public and private projects and establish foundations for implementations in the CodeScene tool [34].[1]

In this study, we modified the algorithm proposed by Avelino et al. [5] to use knowledge models other than the *Degree of Authorship*. In this way, we can investigate the possibility of improving this algorithm considered the most accurate in the aforementioned study.

Furthermore, it is possible to compute the truck factor using information other than those present in *VCS*. Jabrayilzade et al. modified the *Degree of Authorship* to include information about the number of reviews made by the developer, and time spent in meetings about the file's commit [9]. Even though this is an improvement in inferring developers' knowledge, this information is more difficult to obtain, which makes it difficult to generalize the use.

3. Proposing knowledge models

A ground truth with data on source code experts is required to analyze which variables relate to source code knowledge and propose models of expertise. We build this oracle by extracting data from open-source and industrial projects. This ground truth is composed of the developers' knowledge in source code files and variables from development history.

3.1. Target subjects

We selected open-source repositories from the GitHub platform.[2] To select these repositories, we adopted a similar procedure to other studies that investigate GitHub data [5]. First, for six popular programming languages[3] – Java, Python, Ruby, JavaScript, PHP, and C++ – we selected the 50 most popular repositories as indicated by their number of stars. After cloning the repositories, we performed a repository filtering step based on: the number of commits, number of files, and number of developers. As in a previous work [5], for each language, we removed repositories in the first quartile of the distribution of each metric, resulting in the intersection of the remaining sets, i.e, repositories with few commits, files, and developers were removed. Table 1 shows the first quartile of each of the three metrics for each programming language.

[1] https://codescene.com

[2] https://github.com/

[3] Six popular programming languages https://octoverse.github.com/#top-languages.

Table 1
First quartiles of filtering metrics, for each language.

Language	Commits	Files	Developers
Python	510.75	87.50	45.25
Java	829.25	318.00	39.25
PHP	823.50	89.00	97.50
Ruby	1,650.25	152.75	198.25
C++	2,010.25	706.00	113.50
JavaScript	1,455.75	113.25	129.25

Table 2
Target open source repositories.

Language	Repos	Devs	Commits	Files
Python	25	18,936	192,587	39,154
Java	17	5,733	138,473	62,429
PHP	16	9,802	144,092	29,902
Ruby	25	38,036	605,546	88,869
C++	14	11,467	350,345	72,991
JavaScript	14	10,319	109,541	21,477
Total:	111	94,293	1,540,584	314,822

Additionally, we discarded repositories whose history suggests that most of the software was developed outside of GitHub by removing repositories where more than half of its files were added in a few commits. As few commits, we considered the outliers of the distribution of the number of files added in each commit of a repository; we discarded repositories if most of their files (>50%) were added by the set of outliers commits, following the procedure done by Avelino et al. [5]. The resulting dataset is composed of 111 GitHub repositories distributed over the six programming languages, which have a relevant number of developers, files, and commits. Table 2 summarizes the characteristics of these 111 repositories.

We also used data from two industrial projects by Ericsson,[4] both developed in the Java programming language. The development history of Project #1 has 74,078 commits, 17,329 files, and 513 contributors. Project #2 has 26,678 commits, 15,930 files, and 262 contributors. Therefore, the two industrial projects are relevant according to the criteria for selecting the open-source repositories.

3.2. Oracle construction

Extracting Development History. The data was extracted by collecting the commits from the *master/default-branch* from the repositories. First, we ran `git log -no-merge -find-renames`[5] to extract data from the commits logs. From each commit, we extracted: (1) changed files; (2) name and email of the commit's author; (3) type of the change: *addition*, *modification*, or *rename*. Then, we discarded files that did not contain source code and third-party libraries using the *Linguist* tool.[6] Finally, we handled developer aliases by following the same procedure adopted in other works [5,8,13,35]. We merged developer histories with the same email, and used *Levenshtein* distance maximum modification threshold to merge histories of developers with similar names [16,36].

Generating Survey Sample. We used information extracted from development history to create sample pairs *(developer, file)* for each repository. These samples are necessary to elaborate the survey we applied. These were created by performing the following steps for each repository:

1. We randomly selected a file and retrieved the list of developers who touched (created or modified) it.
2. We discarded the file if at least one of these developers reached the maximal limit (5) of files we plan to send to a developer (*file_limit*), asking about his/her expertise on the file. Otherwise, we added the file to each developer's list.
3. We repeated steps 1 and 2 until there were no more files to be verified.

Step 2 warrants that a file will be added to the sample only if it is possible to add all developers who modified it. This step aims to maximize the possibility of obtaining answers from all developers who touched a file. We established a *file limit* of five, seeking to not discourage developers from responding to the survey or responding randomly.

Ultimately, we created 20,564 pairs for the open-source repositories (including 7803 developers, 2.64 files per developer on average) and 394 pairs for the Ericsson repositories (including 92 developers, 4.34 files per developer).

Sending the Survey. The developers were invited via email to evaluate their knowledge of each of the files on their list. To do so, we asked them to use a scale from 1 (one) to 5 (five), where *(1) means you have no knowledge about the file's code; (3) means you would need to perform some investigations before reproducing the code; and (5) means you are an expert on this code*. For the open-source repositories, we sent 7803 emails, and 501 responses were received, representing a response rate of 7%. For the private repositories, we sent 92 emails, and 38 responses were received, representing a response rate of 41%.

Processing the Answers. We process the answers by classifying each pair *(developer, file)* into one of two disjoint sets: *declared experts* (O_m), and *declared non-experts* ($O_{\overline{m}}$). A *declared expert* is a developer who claims to know more than 3 (three) in a file; otherwise, the developer is a *declared non-expert*. Fig. 1 shows the distribution of responses in the public and private datasets. The public dataset is composed of 54% of *declared experts* and 46% of *declared non-experts*, and the private dataset is composed of 47% of *declared experts* and 53% of *declared non-experts*. As we can see, the proportion of declared experts and declared non-experts sets is close in both datasets. The complete dataset comprises 1187 evaluations, with an average of 2.51 files per developer, with this distribution of the number of evaluations per developer having the first, second, and third quartiles of 1, 1, and 3.

3.3. Development variables

Extracting Variables. We selected 12 variables (or features) that can be extracted from the development history. These variables and their meanings are shown in Table 3. Subsets of these variables are explored in other studies. Table 4 lists studies that analyzed these variables in the context of inferring source code knowledge. As shown, *NumCommits* is the most explored.

The variables *Adds*, *Dels*, *Mods*, and *Conds* are extracted using the command `git diff`.[7] In this work, a modification is defined as a set of removed lines followed by a set of added lines of the same size [37]. We use *Levenshtein* distance [36] to identify which pairs *(remove, add)* are modifications between two versions of the files. The algorithm takes the removed and added lines as input and returns a value that represents the number of characters that must be modified to transform the removed line into the added line. In this work, a change is a modification if the returned value is less than a certain percentage (threshold) of the size of the removed line. A threshold of 40% was used, as suggested by Canfora et al. [38].

[4] https://www.ericsson.com/en
[5] https://git-scm.com/docs/git-log/1.5.6
[6] https://github.com/github/linguist/blob/master/lib/linguist/languages.yml
[7] https://git-scm.com/docs/git-diff

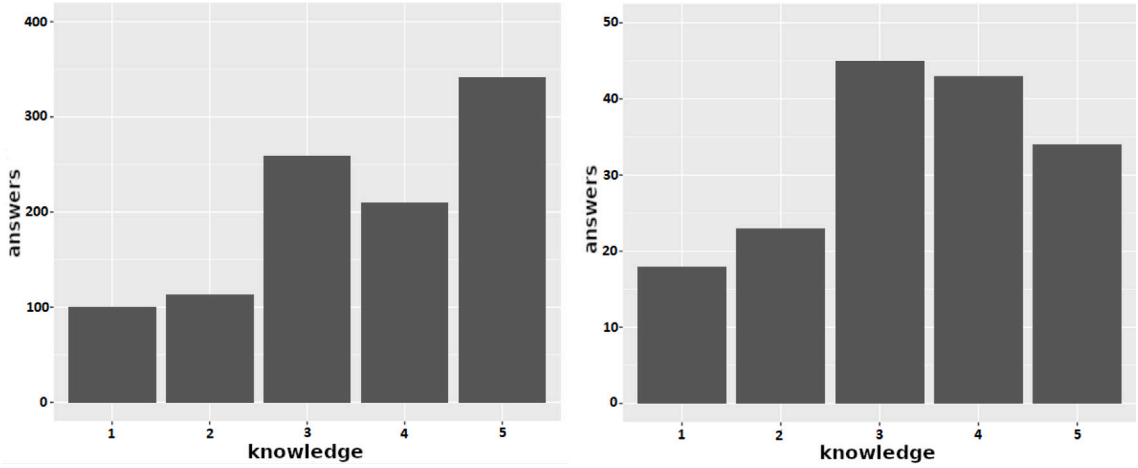


Fig. 1. Responses distribution in the Public And Private dataset.

Table 3
Variables extracted from the development history. The variables description are given considering a developer d and a file f in its last version.

Variable	Meaning
Adds	Number of lines added by a developer d on a file f
Dels	Number of lines deleted by a developer d on a file f
Mods	Number of lines modified by a developer d on a file f
Conds	Number of conditional statements added by a developer d on a file f
Amount	Sum of number of added and deleted lines of a developer d on a file
FA	First Authorship. Binary variable that indicates whether a developer d added the file f to the repository
Blame	Number of lines authored by a developer d that are in a file f
NumCommits	Number of commits made by a developer d on a file f
NumDays	Recency of Modification. Number of days since the last commit of a developer
NumModDevs	Number of developers that committed on the file f since the last commit of a developer d
Size	Number of lines of code (LOC) of the file f
AvgDaysCommits	Average number of days between the commits of a developer d on a file f

Table 4
Extracted variables and related studies.

Variable	Also used in these studies
Adds, Dels	[37,39–41]
Mods, Conds, Amount	[37,39,40]
Blame	[8,22,31,42]
FA	[6]
NumCommits	[6,8,11,22,29,37,39,40,43] [7,24,25,31,44,45]
NumDays	[8,22,37,39,40,44]
NumModDevs	[6,37,39,40]
Size	[8,43]
AvgDaysCommits	[29]

3.4. Compared techniques

This section describes the techniques used in this study to identify code experts.

Commits. This technique counts the number of commits as a measure of knowledge that a developer has in a code file. The idea is that a developer gains knowledge in a file by modifying it. In other words, more commits are interpreted as more knowledge. This technique is widely used in the context of developer expertise (Table 4). It is applied individually [8,31], as well as combined with other techniques [6,44]. In this study, we use the number of commits individually as an expert identification technique and refer to it as *NumCommits*.

Blame. This technique infers the knowledge that a developer has in a file by counting the number of lines added by him in the last version of the file. Blame-like tools such as `git-blame` command are used to identify the authors of each line.[8] As in other works [8,22,31], we use the percentage of lines associated with a developer as a measure of knowledge.

Degree of Authorship (DOA). Fritz et al. [6] proposed that the knowledge on a source code file depends on the file's authorship, number of contributions, and number of changes made by others. The authors combined these variables into a linear model called *Degree of Authorship* (DOA). The weights associated with each variable in this model were defined through an empirical study based on the development history data from two Java systems. The authors used multiple linear regression to find the weights associated with each variable, and they claim that this model is robust enough to be applied in different analyses. In the studies by Avelino [5] and Ferreira [4], *DOA* is used to classify developers into experts using classification thresholds. The *DOA* value that a developer d has in the version v of a f file is calculated by the following equation:

$$\mathbf{DOA}(\mathbf{d}, \mathbf{f}(\mathbf{v})) = 3.293 + 1.098 * FA + 0.164 * DL - 0.321 * ln(1 + AC) \quad (1)$$

where,

- **FA**: 1 if developer d is the creator of the file f, 0 otherwise; **DL**: is the number of changes made by developer d until version v of file f; **AC**: is the number of changes made by other developers in the file f up to version v.

[8] https://git-scm.com/docs/git-blame

Table 5
Coefficients for the linear model proposed.

	Estimate	Std. error	p-value
Intercept (C)	5.28223	0.19685	<2e−16
Adds	0.23173	0.02724	<2e−16
FA	0.36151	0.10037	0.000329
NumDays	−0.19421	0.02400	1.46e−15
Size	−0.28761	0.03020	<2e−16

Residual std. error: 1.152; R-squared: 0.2256;
F-statistic: 86.11; **p-value**: <2.2e−16

Degree of Expertise (DOE). It's not uncommon for applications to require not only the identification of experts but also a measure of their level of expertise for ranking proposes [46]. To address this need, we used the development variables (Table 4) *Adds*, *FA*, *Size*, and *NumDays* to propose a linear model that calculates the knowledge of developers in source code files. The rationale for these choices is given in Section 3.6, where we present the results of the *RQ1*. We named this model *Degree of Expertise* (DOE).

Following the same idea as the *Degree of Authorship* (DOA), we propose the use of linear regression for this. In defining the linear model, log transformation was used in the variables *Adds*, *NumDays*, and *Size*, to deal with the difference in scales. The knowledge of a developer d in the version v of a file f is given by Eq. (2). To use this model in Truck Factor applications we need values for its coefficients. We obtained final values using all available data to train the model, presented in Table 5.

$$\begin{aligned}\mathbf{DOE}(\mathbf{d},\mathbf{f}(\mathbf{v})) = C + b_0 * ln(1+\mathbf{Adds}^{d,f(v)}) + b_1 * (\mathbf{FA}^{f}) \\ - b_2 * ln(1+\mathbf{NumDays}^{d,f(v)}) - b_3 * ln(\mathbf{Size}^{f(v)})\end{aligned} \tag{2}$$

Machine Learning Classifiers. We also investigate the applicability of machine learning algorithms for identifying source code experts. This proposal comes down to binary classification. From the dataset described in Section 3.2, we labeled each pair *(developer, file)* in non-expert (knowledge 1–3, as their answers in the survey) and experts (knowledge 4–5). We used the same variables of *DOE* as features to train the machine learning models. According to the values of these features, machine learning models can classify a developer as an expert or not from a source code file.

Five well-known machine learning classifiers are evaluated: Random Forest [47], Support-Vector Machines (SVM) [48], K-Nearest Neighbor (KNN) [49], Gradient Boosting Machine [50], and Logistic Regression [51]. We use *10-fold Cross-Validation* with three repetitions to evaluate the performance of the machine learning classifiers. 10-fold Cross-Validation consists of randomly partitioning the data set into ten complementary subsets, and the use of each of these subsets for model training and the rest for model validation. In the end, the fitness measures are joined to obtain a more accurate estimate of the model's performance. This process is repeated three times with different random folds, and the performances are averaged over the runs. We use *F-Score* as a performance measure. To find the best settings for these classifiers, we perform a grid search to adjust the hyper-parameters and find the best settings for each classifier.

3.5. Linear techniques evaluation

To evaluate the performance of *NumCommits*, *Blame*, *DOA*, and *DOE* as classifiers, we need to define a k classification threshold for each one of them. To achieve this, we normalized the values of these metrics to a common range. Then, we tested the same set of thresholds for all metrics to find the best one and compare their performances in identifying experts. For this normalization, we adopted a procedure followed in a previous related work [8].

For this purpose, we set as 1 the expertise of a developer d in file f if d has the highest value for a given technique in f; otherwise, we set a proportional value. For example, suppose that for a given file f developers *d1*, *d2*, and *d3* have a *Blame* value of 10, 15, and 20. Their normalized values for blame regarding the file f are as follows: *expertise(d1, f)* = 10/20 = 0.5, *expertise(d2, f)* = 15/20 = 0.75, and *expertise(d3, f)* = 20/20 = 1. We apply this normalization process to all techniques. After normalization, a developer d is classified as an expert of a file f if its *expertise (d, f)* is higher or equal than a threshold k; otherwise, the developer is considered a *non-expert*. In this example, considering a threshold k = 0.7, developers *d2* and *d3* would be considered experts of the f file accordingly with the *Blame* technique. This normalization is done to facilitate the application of different classification thresholds for all techniques, which have different ranges of values, as explained in the next paragraphs.

Evaluating Performance. We compared the performance of the technique in their best settings. For the linear techniques, this requires setting the classification threshold k that maximizes the correct identification of file experts. Thus, we first compute the performance of each technique by adopting 11 different thresholds, i.e., by varying the threshold k from 0 to 1, under 0.1 steps. At the end of this process, the best performance of the linear techniques is obtained together with their associated thresholds k. For each threshold, we use *10-fold Cross-Validation* to compute the *F-Score* of the techniques by applying the following equations:

$$Precision(k) = \begin{cases} \frac{|(d,f)\in O_m \ | \ expertise(d,f) > k|}{|\ expertise(d,f) > k\ |}, & \text{if } k = 0 \\ \frac{|(d,f)\in O_m \ | \ expertise(d,f) \geq k|}{|expertise(d,f) \geq k\ |}, & \text{otherwise} \end{cases} \tag{3}$$

$$Recall(k) = \begin{cases} \frac{|(d,f)\in O_m \ | \ expertise(d,f) > k|}{|O_m|}, & \text{if } k = 0 \\ \frac{|(d,f)\in O_m \ | \ expertise(d,f) \geq k|}{|O_m|}, & \text{otherwise} \end{cases} \tag{4}$$

$$F-Score(k) = 2 * \frac{Precision(k) * Recall(k)}{Precision(k) + Recall(k)} \tag{5}$$

where O_m is the set of declared experts, as we inferred from the survey responses ().

Thresholds Calibration. Fig. 2 shows the *F-Score* results for each linear technique at the analyzed thresholds under the public and private datasets. As we can see, *Blame* reaches its best performance when we use the lowest possible threshold ($k = 0$), in both datasets. In other words, by adopting $k = 0$, *Blame* reaches its best performance when it is used to classify as an expert any developer who has at least one line of code in the last version of the file. Similarly, *NumCommits* achieves the best performance by adopting the thresholds $k = 0.1$ and $k = 0.3$, in the public and private datasets, respectively. The low thresholds indicate that both techniques perform better by relying on minimal changes to consider developers as file experts. On the other hand, *DOA* has its best performance with a threshold $k = 0.1$. on public data; and $k = 0.7$, on private data. Finally, *DOE* had the best results with high thresholds of $k = 0.7$ with public data, and $k = 0.8$ with private data.

3.6. Results

This section presents the results of the two research questions proposed in the first study. We present the correlation analysis between the extracted variables and developers' knowledge. Next, we present the performance of the techniques in our two datasets.

***RQ1:* How do repository-based metrics correlate with developer's knowledge?**

We analyze the correlations between the development variables and the developers' knowledge, as obtained in the survey. In the remainder of this paper, the survey responses will be represented by a variable named *Knowledge*. We answer *RQ1* and, consequently, identify which variables could be part of a knowledge model.

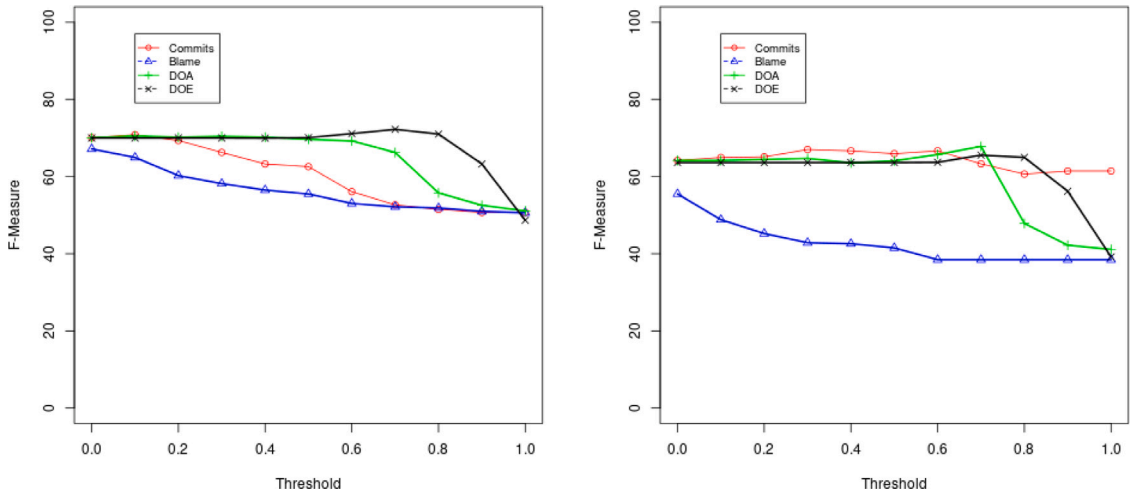


Fig. 2. Performance at analyzed thresholds using Public and Private data.

Table 6
Correlation of extracted variables with *Knowledge*.

Variable	Corr. with Knowledge	p-value
NumDays	−0.24	<0.001
Size	−0.21	<0.001
NumModDevs	−0.21	<0.001
Mods	0.01	0.515
Dels	0.02	0.369
Cond	0.19	<0.001
NumCommits	0.20	<0.001
AvgDaysCommits	0.21	<0.001
Amount	0.28	<0.001
Blame	0.29	<0.001
Adds	**0.31**	<0.001
FA	**0.33**	<0.001

Table 6 shows the directions and intensities of the correlations between the extracted variables and the *Knowledge* variable by applying Spearman's *rho*. We consider results to be statistically significant when $p < 0.05$. Even though *NumCommits* is the most used variable for inferring a developer's knowledge, it does not show the strongest positive correlation with the *Knowledge* variable. The variable with the highest positive correlation is *FA*, closely followed by *Adds*. This suggests that a more fine-grained measure of changes like *Adds* can be more important than *NumCommits* to compose a knowledge model. Finally, the variable that shows the highest negative correlation is *NumDays*, followed by *NumModDevs* and *Size*. These results reinforce the importance of recency for inferring knowledge about a source code.

We also analyze the correlation between the extracted variables using Spearman's *rho*. These correlations are represented in Fig. 3, where colors close to 1 and −1 represent higher positive and negative correlations, respectively. To interpret the correlations in terms of *weak*, *moderate*, and *strong*, we used the interpretation commonly used in some works [52,53].

As expected, *NumCommits* has a moderate correlation with *Adds*, *Dels*, *Mods*, *Amount*, and *AvgDaysCommits* (*rho* ≥ 0.5). Therefore, any one of these five variables can be used to measure the number of changes made by a developer throughout the history of a file. The variables *NumModDevs* and *NumDays* also show a moderate positive correlation (*rho* ≥ 0.5) with each other.

> **Summary of RQ1:** *First Authorship* (FA) has the highest positive correlation with knowledge in source code files, suggesting that file creators tend to have high knowledge of the files they create. On the other hand, *Recency of Modification* has the highest negative correlation, indicating that a developer's knowledge of a source code file decreases over time, since its last modification.

Variable Selection Rationale. In previous research, Avelino described how the variables *file size* and *recency* influence developer's knowledge on source code files [8]. Thus, models that use this information tend to be more accurate. This previous finding is reinforced by the results in Table 6, where these variables achieved the highest negative correlation with *Knowledge*. Therefore, we include the variables *Size* and *NumDays* in the proposed machine learning models. Considering the variables *Adds*, *Amount*, and *Blame*, we choose the variable *Adds*, as it has the highest positive correlation with *Knowledge* among the three. No more than one of the three variables is chosen because these variables are conceptually related, as commented in Section 3.3.

The variable *NumCommits* is not included in the models. It can be viewed as just a macro way of accounting for changes made by a developer to a source code file, and the variable *Adds* has already been chosen since it has a higher positive correlation with *Knowledge*. Additionally, *Adds* has a moderate correlation with *NumCommits* (*rho* ≥ 0.5, Fig. 3). For a similar reason, *AvgDaysCommits* is not included in the model. We also add the variable *FA*, which has the highest positive correlation with *Knowledge* among all variables. The variable *NumModDevs* is not included due to its moderate correlation with *NumDays* (*rho* ≥ 0.5).

***RQ2:* How do machine learning classifiers compare with traditional techniques in identifying source code experts?**

After identifying and selecting the variables with the highest correlations with source code knowledge, we train the Machine Learning (ML) models and the Linear Regression model (DOE) using them as features. Next, we compared the performance of these models in the identification of experts. Table 7 presents the performance of linear techniques and machine learning classifiers, in the two analyzed scenarios.

Among the linear techniques, using the public dataset (Table 7 - **Public**), *DOE* had the best *F-Score* (72%) closely followed by *DOA* and

Table 7
Performance of linear and machine learning techniques.

		Public			Private		
	Metrics	Precision	Recall	F-Score	Precision	Recall	F-Score
Linear Metrics	DOE	0.60	0.91	0.72	0.50	0.95	0.64
	DOA	0.55	0.97	0.70	0.61	0.77	0.68
	Commits	0.60	0.87	0.70	0.55	0.86	0.67
	Blame	0.56	0.83	0.67	0.50	0.62	0.55
ML Classifiers	Random Forest	0.73	0.74	0.73	0.59	0.62	0.60
	SVM	0.71	0.75	0.73	0.63	0.60	0.61
	KNN	0.71	0.71	0.71	0.70	0.69	0.67
	GBM	0.73	0.73	0.73	0.65	0.58	0.60
	Logistic Regression	0.69	0.75	0.72	0.70	0.51	0.58

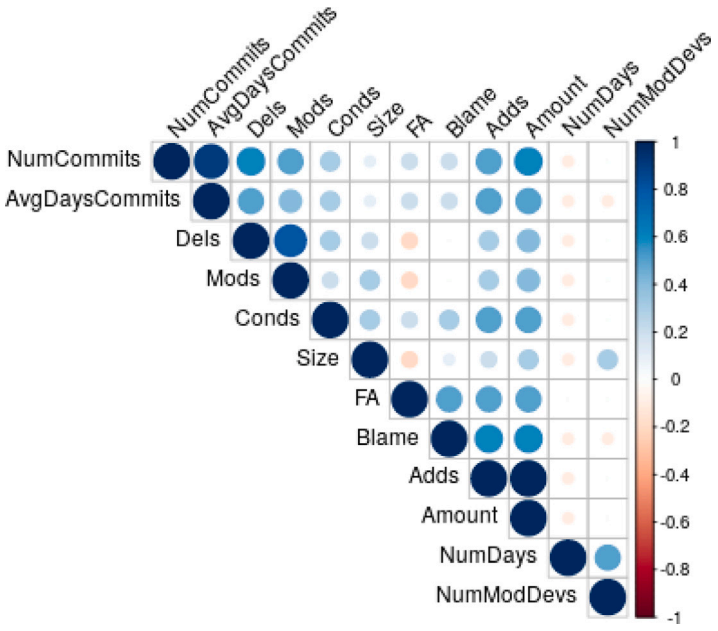


Fig. 3. Correlation between dependent variables.

NumCommits (70%), and *Blame* had the lowest *F-Score* of 67%. The technique with the best *Recall* was *DOA* (97%), and the one with the lowest *Recall* in the same scenario is *Blame* (83%). Finally, the techniques with the highest *Precision* are *DOE* and *NumCommits* (60%), while *DOA* has the lowest *Precision* (55%). Using the private dataset (Table 7 - **Private**), *DOA* had the highest value for *F-Score* (68%), closely followed by *NumCommits* (67%). As in the other scenario, *Blame* had the worst *F-Score* (55%). The best *Recall* was from *NumCommits* (86%), with *Blame* presenting the lowest *Recall* (62%). Regarding *Precision*, *DOA* achieved the best result (61%), and *Blame* the lowest one (50%).

Considering the machine learning models, using the public dataset (Table 7 - **Public**), *Random Forest*, *SVM*, and *GBM* had the best *F-Score* (73%), with the other two classifiers reaching close values. The classifiers with the highest *Recall* were *SVM* and *Logistic Regression* (75%), and *KNN* obtained the lowest result (71%). In terms of *Precision*, *GBM* and *Random Forest* have the highest values (73%). The lowest *Precision* is from *Logistic Regression* (69%). With the private dataset (Table 7 - **Private**), the classifier with the highest *F-Score* was *KNN* (67%) and *Logistic Regression* had the lowest one (58%). The best *Recall* was also obtained by *KNN* (69%) and *Logistic Regression* had the lowest one (51%). Finally, the best *Precision* was achieved by *Logistic Regression* and *KNN* (70%). *Random Forest* had the lowest *Precision* (59%).

Summary of RQ2: In the public dataset, in general, machine learning classifiers outperformed the linear techniques (*F-Score* = 71% to 73%). In the private dataset, this advantage is not clear, with F-Score ranging from 55% to 68% for the linear techniques and 58% to 67% for *ML* techniques.

3.7. Discussion

We start by highlighting our key findings.

1. The linear techniques (*DOE*, *DOA*, *Commits*, and *Blame*) tend to have low precision, but high recall in the two analyzed datasets. In other words, they tend to positively classify developers as experts, but at the cost of a significant number of false positives.
2. Regarding the linear techniques, *DOE* has the best performance using the largest dataset, and *DOA* outperformed using private data.
3. The *ML* classifiers have a distinct performance in the investigated datasets. However, we must highlight the size difference between both datasets: there are 1024 data points in the public dataset and only 163 in the private one. This difference can have a major impact on algorithms that include a training phase. In other words, *ML* techniques tend to improve their performance

as the size of the training set increases, up to a certain point of stability [54]. For this reason, we argue the public dataset results are more representative of the performances of such techniques.
4. Considering the public dataset, the best *ML* classifiers are *Random Forest*, *SVM*, and *Gradient Boosting Machines*. On the other hand, in the private dataset, *K Nearest Neighbors* reached the best performance.
5. Even though there are linear techniques with similar *F-Score*, the machine learning classifiers show a better balance between *Precision* and *Recall*.

The linear techniques and the machine learning classifiers achieved similar performance, particularly if we analyze *F-Score*. For this reason, the choice of the best technique depends on the user's tolerance to false positives and false negatives. Particularly, we envision two application scenarios. First, when the application requires finding few or even just one expert, such as selecting a person to successfully onboard new developers [25], machine learning classifiers are more recommended as they are more precise.

An example where greater precision is necessary, is the identification of the concentration of knowledge in the project, a situation related to the Truck Factor measurement, as explained in our practical application in the next section. Metrics that overestimate a project's Truck Factor, a measure of its dependence on individual developers, can give a false sense of security, suggesting that the project can rely on more developers than it does [55]. Second, when the application requires additional developer expertise, such as during the evolution or maintenance of non-complex features [17], tolerating some false positives becomes more justifiable. In these scenarios, employing linear techniques is often beneficial, as they offer a wider selection of developers possessing greater knowledge in the files.

4. Practical application: Improving a truck factor algorithm

In this section, we describe a practical application of the new models proposed in the first study, aiming to improve the results of an algorithm for Truck Factor estimation. For this purpose, we decided to investigate the performance of the algorithm proposed by Avelino [5] to compute the Truck Factor of a project using two different models: *Degree of Expertise* (DOE) and *Random Forest* (RF). We chose these models because they had the best performance between the linear techniques and the machine learning classifiers in the largest dataset used in Section 3.6.

We evaluate the modified algorithm in two scenarios. In a quantitative analysis, evaluating its performance using an extended dataset of Avelino's study [5], where missing *Truck Factor Developers* are included in the set of TF Developers generated by the algorithm using *DOA*. In a second qualitative analysis, we applied a survey with development leaders of an industrial project and compared the accuracy of *Truck Factor Developers* listings generated by the evaluated models.

4.1. Expert identification and truck factor

We start with a description of the Truck Factor algorithm and how we modified in this study. The Avelino's Truck Factor algorithm follows a greedy approach that relies on an authorship metric to identify the top authors of a system and iteratively estimate the impact of removing the top developers. First, the experts of each file in the project are identified by adopting the *Degree of Authorship* (DOA) metric. Then, the algorithm iteratively removes the developer who is the expert in the largest number of files and checks how many files become abandoned, i.e. files without experts after the removal. When more than half of the projects' files have no expert, the algorithm stops, returning the Truck Factor estimation of the number of experts removed. This procedure is represented in Algorithm 1.

Therefore, as a practical application of the results of the first study (Section 3.6), in this section, we investigate the results of modifying the original algorithm, that adopts the *DOA* model, by replacing it with *DOE* and *Random Forest* models for classifying developers into experts (line 1). For *DOE*, we used the coefficients presented in Table 5 and the best threshold pointed in Section 3.4. Regarding Random Forest, we also obtained a final model by training the model with all data. We checked Random Forest's performance by running the procedure using *10-Fold Cross-Validation* with all the data, and the results were basically the same using only the public data: *Precision = 72%*, *Recall = 73%* and *F-Score = 72%*.

```
1  E ← getExperts();
2  F ← getFiles(E);
3  tf ← 0;
4  while E ≠ ∅ do
5  |  coverage ← getCoverage(F, E);
6  |  if coverage ≤ 0.5 then
7  |  |  break;
8  |  end
9  |  E ← removeTopAuthor();
10 |  tf ← tf + 1;
11 end
12 return tf;
```

Algorithm 1: High-level pseudo-code of the algorithm proposed by Avelino for computing the Truck Factor of a software project.

4.2. Data gathering

In this section, we describe how we collected the data for the two analyses we performed. First, we describe how we extract data from public projects and the private one. Then, we describe how we mined the comments that pointed to missing important developers in the list of Truck Factor Developers generated in the previous study, and how we designed and applied the survey with the private project development leaders.

Extracting Development Data. In the original study, Avelino et al. computed the truck factor of 133 open-source projects using an algorithm that uses *DOA* to identify expertise [5]. The list of projects can be found in the paper.[9] We obtained these repositories in the versions used by Avelino to compute the truck factor using *DOE* and Random Forest for each repository, we cloned it and checkout using the command `git checkout ‘git rev-list –1 --before=date branchName‘`. Thus, using the information provided, we retrieved the projects at the same version used in Avelino's study. However, we were unable to checkout three projects used: *ass/sass*,[10] *driftyco/ionic*,[11] *cucumber/cucumber*,[12] probably due to internal changes in their historic structure. Therefore, we are left with 130 open-source projects for truck factor analysis. Table 8 shows the number of repositories for each programming language, the number of commits, files, and developers that were part of the analysis.

Additionally, we also investigate the Truck Factor of a software project from a Brazilian health tech company.[13] The selected project is a *web system* developed in the *Java* programming language, using a framework developed by the company. The project has nine years of development history, and we analyzed *12,996 commits*, from *6369 files*, with *94 contributors*.

To extract the development histories of these projects, we used the same strategy described in Section 3.2. For the private project, we handled part of the file selection manually, as *Linguist* ended up excluding

[9] Publicly available http://aserg.labsoft.dcc.ufmg.br/truckfactor.
[10] https://github.com/sass/sass
[11] https://github.com/driftyco/ionic
[12] https://github.com/cucumber/cucumber
[13] https://inas.maida.health/

Table 8
Target repositories.

Language	Repos	Devs	Commits	Files
Python	22	7,682	218,673	12,988
Ruby	31	17,672	254,266	21,009
JavaScript	22	5,259	88,641	8,707
PHP	17	3,096	116,596	18,962
Java	20	3,980	287,431	92,138
C/C++	18	16,448	725,259	61,350
Total	130	54,137	1,690,866	215,154

some files with extensions used by the internal framework used by the company. We manually added the files with extensions *'jhm.xml'* and *'hbm.xml'*, which are used by the framework for creating screens and configuring persistence models respectively. We also discard some commits related to code migrations and systematic changes caused by library version updates. We understand that these changes do not require significant knowledge from developers and may potentially introduce errors in results, such as considering a developer with very few contributions as one of the project's main experts. Thus, we performed a manual inspection of the commits and decided to remove commits with more than 90 files changed. Similar filtering has been done in related work to detect migrations in projects [5,13]. This threshold works well for our analysis, however, we emphasize its context-specific nature and do not claim generalizability.

Extracting Previous Survey Comments. In Avelino's study, the authors applied a survey with the developers of the analyzed projects [5]. The survey sought to validate whether the *Truck Factor Developers* identified by the algorithm were in fact the main developers of the project and whether their departure would bring serious problems to the project, validating the Truck Factor estimation. This survey was applied by opening GitHub issues in the target projects.[14] Despite the majority of participants fully or partially agreeing with the listings presented, some answers also included important developers who contributed more recently to the projects. For example, comments such as: "*On the ipython/ipython repo, yes, though I would add Matthias Bussonier who is the expert on many pieces of the code.*", or "*I would have added @DaySˆ and @WonderCsabo as main developers.*"

We looked for similar comments on the provided answers for the original study. Thus, we manually reviewed each answer, searching for comments that claimed deficiencies in the list of developers presented and pointed to missing developers. We identified 15 comments present in 14 projects, pointing to 36 missing developers from the presented lists. However, we decided to only consider comments from project contributors who were active at the time of the comment. We use the same inactivity threshold of one year without commits, used in a study with public projects [13]. By applying this filter, we excluded one comment present in *silexphp/Silex*,[15] leaving a total of 14 comments in 13 projects, citing 34 developers, as we couldn't find the comment authors contribution.

As in Avelino's study [5], we relied on respondents' opinions to verify the ability of our techniques to include these missing developers. We consider the union of the list of *Truck Factor Developers* generated by the algorithm using *DOA* plus the list of developers named in the comments as ground truth to verify the performance of the Truck Factor algorithm using the proposed models. To illustrate this evaluation, Fig. 4 presents a project with *n* contributors. Using *DOA*, the algorithm points as *Truck Factor Developers dev1* and *dev2*. *Dev3* and *dev4* are pointed out by other contributors as important developers who should also be on *Truck Factor Developers*. Using the union of these lists as ground truth, we can evaluate *Precision*, *Recall* and *F-Score* of the *Truck Factor Developers* pointed by the algorithm using *DOE* and *Random Forest*.

In addition to the 15 comments mentioning missing Truck Factor Developers, we were able to identify 20 comments in 18 projects that pointed to the lack of the recency factor in the results. We can cite as an example: "*I would say that this list is kind of old? I'm in the community for a while and never seem to most of those guys. Maybe we might need another metric other then just files updated?*" in *fog/fog* survey,[16] or "*I'm definitely not qualified to be giving advice in this topic, but it could be a good idea to look at recent history as well.*" in *dropwizard/dropwizard* survey.[17] These comments were not used in our evaluations, but they demonstrate the lack of the recency factor in the computation of these Truck Factors using the original approach.

4.3. Survey design and application:

In addition to the quantitative analysis, we also developed a qualitative analysis comparing the truck factors of a private company project using Avelino's algorithm with *DOA*, *DOE*, and *Random Forest*. After computing the Truck Factor with these three models, an online survey was administered to six development leaders familiar with the project. Two possess more than five years of project experience, one between two and three years, and three between one and two years.

In the survey introduction, we provide a brief explanation of the Truck Factor concept and the survey's goal. We also make clear the absence of any professional impact on the answers given. After that, we introduce the questionnaire with a question about the number of years of experience of the respondent in the analyzed project, followed by the three questions about the truck factor estimations. Following good survey design practices, we ran a feedback loop between the survey author and two other authors of this paper until we reached a consensus, and ensured clarity and consistency in our questionnaire. The three questions and aims are presented as follows.

Question 1: In your opinion, which of the listings below best represents the main developers of the project?

With this question, we aim to verify which of the generated listings of main developers comes closest to the reality of the project. Thus, this question helps to verify if, with our models, the algorithm can identify important developers that the original approach missed.

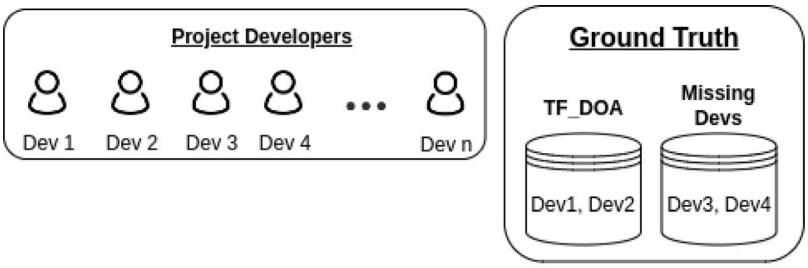
Question 2: Do you agree that if all the developers in the chosen list abandoned the project it will face serious maintenance and evolution problems?

This second question is closer to the truck factor concept. With it, we seek to verify whether the developers from the chosen list are indeed crucial for the continuity of the project. Participants had four response options following a Likert Scale: Strongly Disagree, Disagree, Agree, Strongly Agree. We decided to remove the neutral option to avoid misinterpretation, or its use as a dumping ground or an easy way out by participants, as explained in different studies [56].

Question 3: Would you add any other developers to your chosen listing? Please provide the name(s) of the developer(s) if your answer is yes.

The proposed models may not take into account some aspects and leave out some important developers in the truck factor computation. With this third question, we seek to verify deficiencies in the lists presented. In addition to the three questions, we also left an open space to collect comments on any topic.

[14] Publicly available http://aserg.labsoft.dcc.ufmg.br/truckfactor/survey.html.
[15] https://github.com/silexphp/Silex/issues/1212
[16] https://github.com/fog/fog/issues/3654
[17] https://github.com/dropwizard/dropwizard/issues/1207



	Truck Factor Devs	Precision	Recall	F-Score
DOE	Dev1, Dev2, Dev3	1	0.75	0.86
RF	Dev1, Dev2, Dev3, Dev4, Dev6	0.8	1	0.89

Fig. 4. Computing *Precision* and *Recall* from *Truck Factor Developers List* generated using *DOE* and *Random Forest*.

Table 9
Performances of variations of Avelino's Truck Factor algorithm in the extended ground truth.

Project	Truck factor				Precision		Recall		F-score	
	GroundTruth		DOE	RF	DOE	RF	DOE	RF	DOE	RF
	DOA	Devs+								
fog/fog	12	1	21	10	0.57	1	0.92	0.77	0.71	0.87
chef/chef	6	2	29	20	0.28	0.40	1	1	0.43	0.57
emberjs/ember.js	6	6	30	21	0.40	0.49	1	0.92	0.57	0.67
spotify/luigi	6	1	15	14	0.47	0.50	1	1	0.64	0.67
androidannotations/androidannotations	2	2	6	4	0.67	1	1	1	0.80	1
mailpile/Mailpile	1	1	2	2	1	1	1	1	1	1
ReactiveX/RxJava	1	1	2	1	1	1	1	0.50	1	0.67
ipython/ipython	4	1	10	6	0.40	0.57	0.80	0.80	0.53	0.72
yiisoft/yii2	3	1	9	7	0.44	0.57	1	1	0.62	0.72
elasticsearch/elasticsearch	2	10	11	8	0.72	0.88	0.67	0.58	0.70	0.70
elasticsearch/logstash	1	6	6	4	0.67	1	0.57	0.57	0.62	0.72
Respect/Validation	2	1	5	4	0.60	0.75	1	1	0.75	0.86
dropwizard/metrics	1	1	5	2	0.40	0.50	1	0.50	0.57	0.50
Avg.	**3.62**	**2.62**	**11.62**	**7.92**	**0.59**	**0.74**	**0.92**	**0.82**	**0.69**	**0.74**

4.4. Results

This section presents the results of both quantitative and qualitative evaluations, addressing our third research question *RQ3: Can the proposed models improve a state-of-the-art Truck Factor algorithm?* First, we present the performance results obtained by applying the proposed models and compare these findings with the original Truck Factor study. The comparison is conducted on an extended dataset containing missing developers, as identified by the system experts. Then, we present the survey results with the private project developers.

Mined Comments Missing Developers: Initially, we analyze whether the Truck Factor algorithm using *DOE* and *Random Forest* (RF) manages to list the developers present in our extended ground truth of the 13 selected projects. For this purpose, we compute *Precision*, *Recall*, and *F-Score* of the Truck Factor variations for each project. Table 9 shows Truck Factor using *DOE*, *RF* and our ground truth composed by the original Truck Factor *DOA* plus the missing developers (Devs+), the performances using *DOE* and *RF* in terms of *Precision*, *Recall* and *F-Score*.

Analyzing first the results of *Precision*, the algorithm with *DOE* achieves the best performance (100%) for the *mailpile/Mailpile* and *ReactiveX/RxJava* systems. Conversely, it exhibits the poorest performance (28%) for the *chef/chef* system. Notably, *RF* demonstrates superior overall performance, achieving a *Precision* of 100% in five systems with the worst result being 40% for the *chef/chef* system. The reverse situation happens when we analyze the results of *Recall*. Using *DOE*, the algorithm obtained maximum performance (100%) in nine systems, with the worst result of 57%, while using *RF* it had 100% of *Recall* in six, with a worst result of 50% (*ReactiveX/RxJava* and *dropwizard/metrics*).

As we can observe, when using *DOE*, the algorithm classifies more developers as experts which results in high *Recalls*, with an average of 92%, and lower *Precisions*, with an average of 59%, resulting an average *F-Score* of 69%. Using *RF* we had an average *Precision* of 74% and an average *Recall* of 82%, which, in general, resulted in better *F-Scores*, with an average of 74%. These results show that, in most cases, the variations using the proposed models managed to include the missing developers which are evidenced by the high *Recalls*. However, they also added other developers, which were considered false positives.

We obtain some cases of much larger Truck Factor using the proposed techniques, as is the case of the *chef/chef* ($DOA = 6$, $DOE = 29$, and $RF = 20$) and *emberjs/ember.js* ($DOA = 6$, $DOE = 30$, and $RF = 21$) projects, however, even in cases where the difference is small, as the *mailpile/Mailpile* ($DOA = 1$, $DOE = 2$, and $RF = 2$) and *androidannotations/androidannotations* ($DOA = 2$, $DOE = 6$, and $RF = 4$) the improved algorithms can include exactly the developers mentioned in the comments. In cases where the difference between the Truck Factor of the variations and the *DOA* is quite large like the two mentioned above, we found comments that indicate a greater number of missing important developers, such as: "*Everyone else listed is definitely a 'main developer', but there are quite a few other people who have enough knowledge to navigate and maintain the codebase.*" (*chef/chef*) and "*Losing those four people would certainly have an impact on the project, but I don't think it would be fatal - again, this is due to the growing set of major*

Table 10
Measures on Truck Factors of the 130 open-source projects.

	First quartile	Second quartile	Third quartile	Mean	Std. deviation
DOA	1	2	4	3.97	6.15
DOE	3	7	17	18.01	34.60
Random Forest	2	6	14	14.58	40.10

contributors." (*emberjs/ember.js*). These comments indicate that there are more key developers than those explicitly named in the comments, which could potentially increase the *Precision* results.

Furthermore, using *RF*, we obtain a smaller average Truck Factor of *7.92*, and with *DOE* a higher average of *11.62*. This tendency is reinforced in the computation of the Truck Factor of all the 130 selected open-source projects. Table 10 presents the results of computing the truck factor for the 130 projects. We can see that, in this analysis, the algorithm using *RF* has the mean and the first, second, and third quartiles greater than those reported in the original study using *DOA*, and smaller than those using the *DOE*, which reinforces the aforementioned findings of *Recall* and *Precision* of these techniques.

This large variation found in the Truck Factors computed using *DOE* and *RF* is explained by the presence of outliers (Fig. 5) leaving the distribution quite asymmetric.[18] Looking at why there were these outliers, we take as an example the two biggest ones: *torvalds/linux* and *rails/rails*.[19],[20] Using *DOE* as an example, we have *linux* with a $TF = 311$ and *rails* with a $TF = 185$, and with Random Forest we have *linux* with a $TF = 423$, and *rails* with a $TF = 143$. In comparison, using *DOA* we have *linux* with a $TF = 57$, and *rails* with a $TF = 9$. The exceptionally high Truck Factors observed in these two example projects are attributable to the extended convergence time of the algorithm using our models. In other words, it takes too many iterations, removing experts, until the project's file coverage drops below 50% (Algorithm 1).

Investigating these two projects, we decided to plot the percentage of expertise of each developer to all analyzed files of the project, i.e., the number of files in which each developer is an expert divided by the total number of files analyzed in the project. Fig. 6 shows this expertise percentage distribution for the two projects using the algorithm with *DOE* as an example. We can see that the distribution of expertise in these projects is very uneven. We have a few developers with expertise in a considerable part of the system files, while many others in a very small part. This is consistent with the results of the authorship distribution analysis in the Linux kernel from a previous study [35].

Furthermore, the proposed techniques manage to attribute expertise in cases that *DOA* does not. In Fig. 7, we can see the proportions of developers considered experts in at least one project file of the analyzed projects using the algorithm with the two proposed models. Using *DOE*, most target systems have a significant portion of developers identified as experts in some project file with the first, second, and third quartiles of 56%, 69%, and 81% respectively, and with RF we have 56%, 65%, and 72% for the quartiles respectively. These values are much higher than the quartiles reported by Avelino of 16%, 23%, and 36% [5]. These higher values cause the long tail in the expertise distributions presented, which makes the convergence of the algorithm more difficult in some cases, even with the departure of those with greater knowledge.

Based on the situation exemplified in Fig. 6, a possible solution for these Truck Factors outliers is to define thresholds of expertise to include a developer in the Truck Factor computation. This p threshold would indicate the minimum percentage of files that the developer would have to have expertise to be considered in the calculation of Truck Factor. For example, using the examples previously presented, if we use a low threshold $p = 1\%$, in the computation of *rails* we would end up with a $TF = 97$, excluding 48% of the developers from the computation without the threshold, and with *linux* we would have a $TF = 60$ excluding 85% of the developers of the original computation. This is equivalent to saying that the 50% abandoned files threshold used in the original algorithm is not suitable for all cases, and the ideal algorithm should be flexible according to the distribution of expertise in the project.

Survey Results. We also computed the Truck Factor of the industrial project described in Section 4.3 with variations of Algorithm 1 using *DOA*, *DOE* and *RF*. With *DOA*, following the analysis pattern of the open-source projects, we had the lowest $TF = 4$, with *DOE* we had the highest of $TF = 12$, and with *Random Forest* a $TF = 7$. Analyzing the Truck Factor developers lists generated by each variation, we observed that they were subsets of each other, i.e., the set of Truck Factor developers computed using *DOA* (O_{doa}) is contained in the set with *RF* (O_{rf}) which in turn is contained in the set with *DOE* (O_{doe}) - $O_{doa} \subset O_{rf} \subset O_{doe}$. As in the previous analysis, we have a correspondence between the models, with the proposed models ranking the same developers as *DOA*, but adding other ones that *DOA* cannot identify their expertise.

With the Truck Factor developer listings generated, we applied the survey described in Section 4.2. For *Question 1* – "*In your opinion, which of the listings below best represents the main developers of the project?*" – four participants pointed out the list generated using *DOE* as the most representative, while two participants chose the one using *RF*. This result shows that the proposed techniques were able to include relevant developers that *DOA* was unable to. This is reinforced when we look at the developers included by *DOA* and find that all four have already left the project. Of the four, the developer with the most recent contribution was two years before this computation. Thus, according to the algorithm using *DOA*, the project had experienced a Truck Factor event, or a *Truck Factor Developers Detachment (TFDD)* [13]. In contrast, using both *DOE* and *RF*, the algorithm was able to include important developers who had already left the project, and two relevant developers still active in the project. This corresponds more with the reality of the project, which continues to evolve, and having the two developers appointed as technical references for the project. To better investigate this behavior, we computed the Truck Factor of the project over the past seven years aiming to verify the evolution of the Truck Factor according to the 3 metrics. Starting from the analysis date, we iteratively backward year by year performing the *checkout* of the project and computing the Truck Factor using each of the three models. We can verify in Fig. 8 that by adopting the *DOA* algorithm there was not much evolution of the Truck Factor across the years, always concentrated in the initial relevant developers of the project, unlike the other algorithms that can include new important contributors.

For *Question 2* – "*Do you agree that if all the developers in the chosen list abandoned the project, it would face serious problems in its maintenance and evolution?*" – four of the respondents *Agreed* and two *Strongly agreed* that if the chosen developers abandoned the project, it would enter a Truck Factor situation. This current situation where the project knowledge is hanging on a few developers is reinforced by a respondent's comment: "*I've been at the company for two and a half years and I'm already one of the oldest devs. When I joined I had reference devs to guide me on the project. Devs coming in today should struggle with this.*"

And for *Question 3* – "*Would you add any other developers to your chosen listing? Please provide the name(s) of the developer(s) if your answer is yes.*" – one respondent informed a developer. The informed developer is active in the project, with 306 commits, an expert in 233 files – 4.5% of the project files – according to *DOE*, and an expert in 192 files according to *RF*. For *DOE*, the informed developer is the 13th contributor with the most expertise in project files, just one place after 12 in the calculated Truck Factor, and 10th for *RF*, three places after 7 in the computed Truck Factor, which shows that the models approached the respondent's perception

[18] The truck factor values are publicly available https://doi.org/10.5281/zenodo.10806775.
[19] https://github.com/torvalds/linux
[20] https://github.com/rails/rails

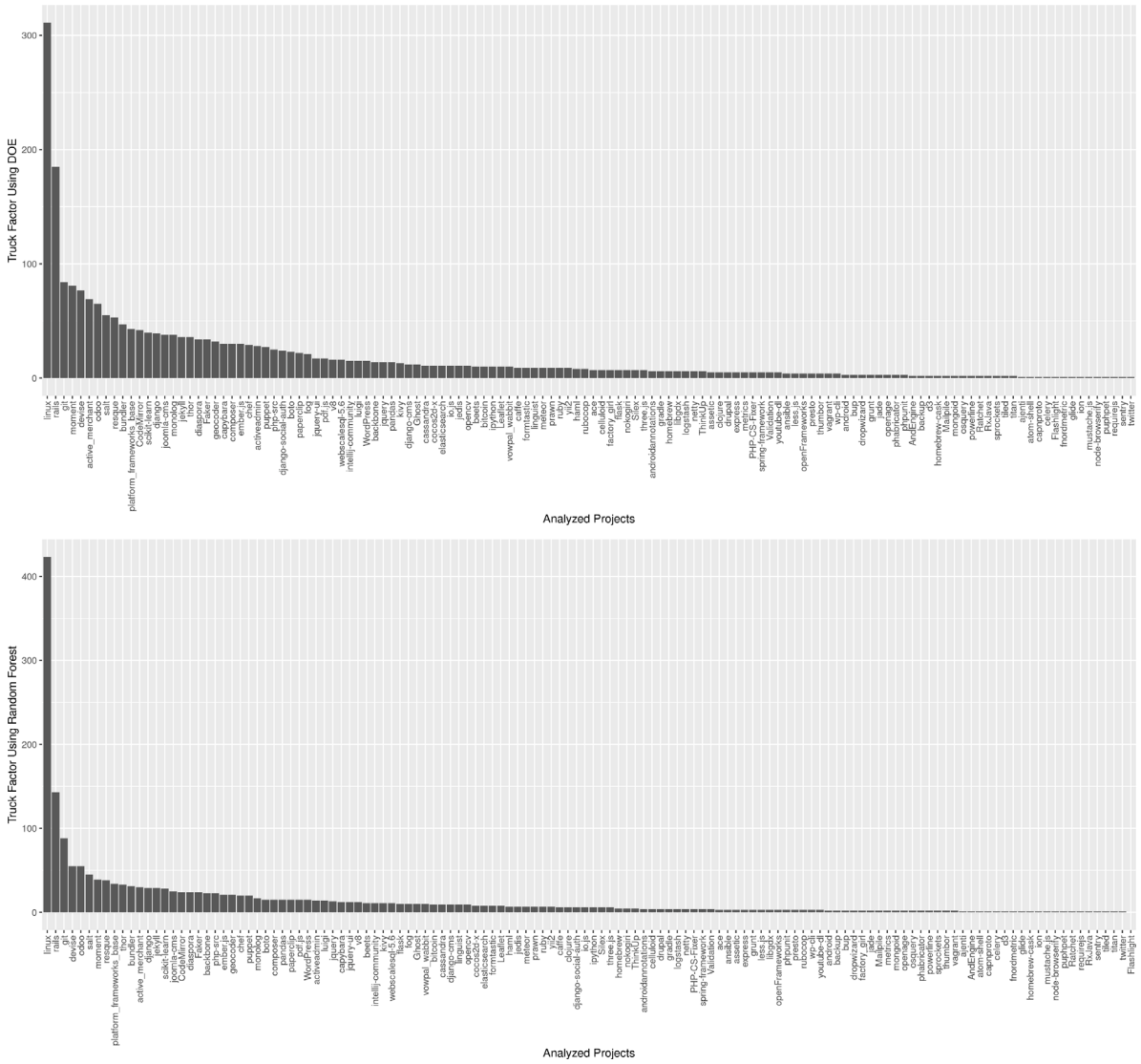


Fig. 5. Truck Factor distribution using *DOE* and *Random Forest* of all 130 selected open-source projects.

Summary of RQ3: Our models improved Avelino's Truck Factor algorithm by including important developers with recent contributions. In a quantitative analysis, the algorithm had an average *F-Score* of 69% using the *Degree of Expertise* and 74% using *Random Forest*. The improvement in the Truck Factor estimation was further validated in a qualitative study, in which the lists of *Truck Factor Developers* generated using our models were perceived as more accurate in a survey with six development leaders of an industrial project.

4.5. Discussion

As in the first study (Section 3.7), we begin by highlighting the main finding of the results presented: *the models proposed in this work can be applied to successfully improve the performance of algorithms to identify knowledge concentration in software projects*. In summary, the quantitative analysis reveals that the algorithm successfully incorporates significant developers previously overlooked when augmented with our models. Moreover, in the qualitative analysis, the computed Truck Factors are discerned as more accurate compared to those derived from the original model.

As we can see, in general, the results of this practical application were consistent with the results presented in the first study, in Section 3.6. When using *DOE*, the algorithm classifies more developers as experts which results in higher *Recalls* and lower *Precisions*, which means more false positives and fewer false negatives. We had the opposite with *RF*, with higher *Precisions* and lower *Recalls* and better *F-Scores*.

In the results of the first question of the survey, the list of Truck Factor developers generated with *DOE*, was perceived as the most accurate by the majority, with others pointing to the list generated by *RF*. If we look at the literature, one of the main applications of Truck Factor is the identification of bottlenecks in development. This happens when a few developers are responsible for a part of the system, and they are overloaded with other activities or are not available. This situation is associated with the community smell known as *code red* and is one of the most widespread community smells [9]. There is an interpretation in the literature that a Truck Factor algorithm that reports fewer false positives is more desirable, as it reduces the risk of giving a false sense of security when the Truck Factor is overestimated [55]. Following this interpretation, the variation of the algorithm that uses *RF* is the most appropriate for practical use, as it presents fewer false positives and greater precision in all studies. According to the best of our knowledge, this is the only interpretation relating Truck Factors results to false

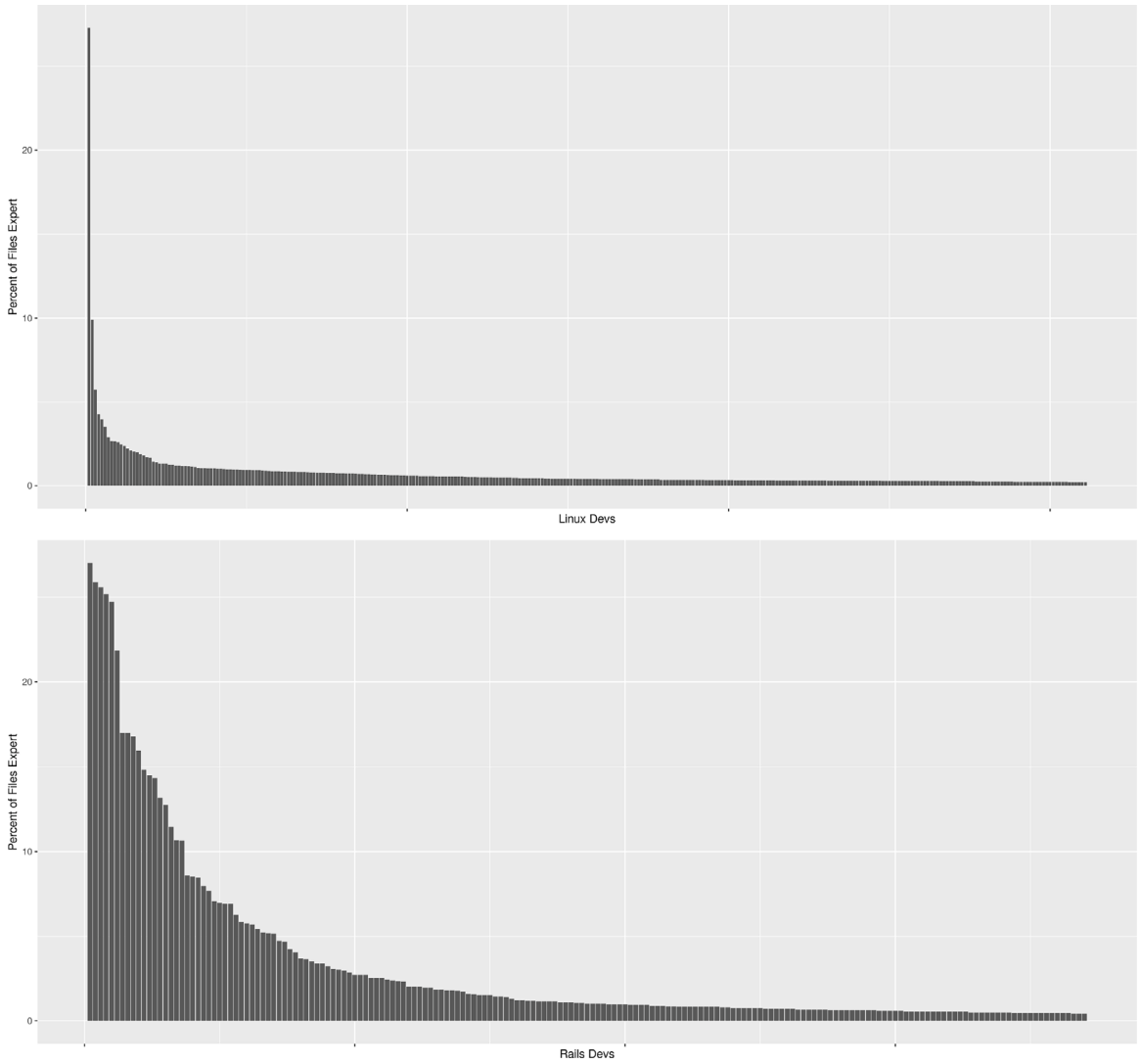


Fig. 6. Expertise distribution in Linux and Rails devs using *DOE*.

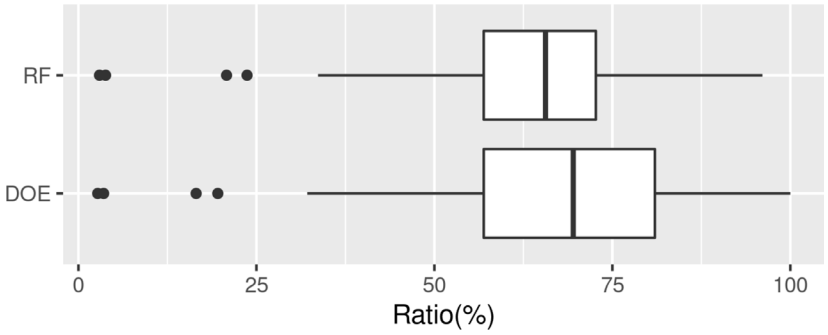


Fig. 7. Proportion of developers ranked as experts.

positive and negative measurements. Still, we are not excluding the existence of applications less sensitive to false positives, a topic that can be explored in future studies.

5. Threats to validity

Construct Validity. For the first survey, some of the developers who participated may have inflated their self-assessment knowledge, due to fear of commercial use of this information. To address this threat, we clarified in the survey that their answers would be used only for academic purposes. We did a similar procedure in the second survey to deal with concerns about choosing certain Truck Factor developers. There is also the possibility of random responses in the first survey. To mitigate this risk and enhance the response rate, we include a small number of files for developers to assess their knowledge, with an average of 2.64 and 4.34 for open-source and private projects respectively. The size of the questionnaire is a demotivating factor for the quality of responses in a survey [57]. Related to this threat, there

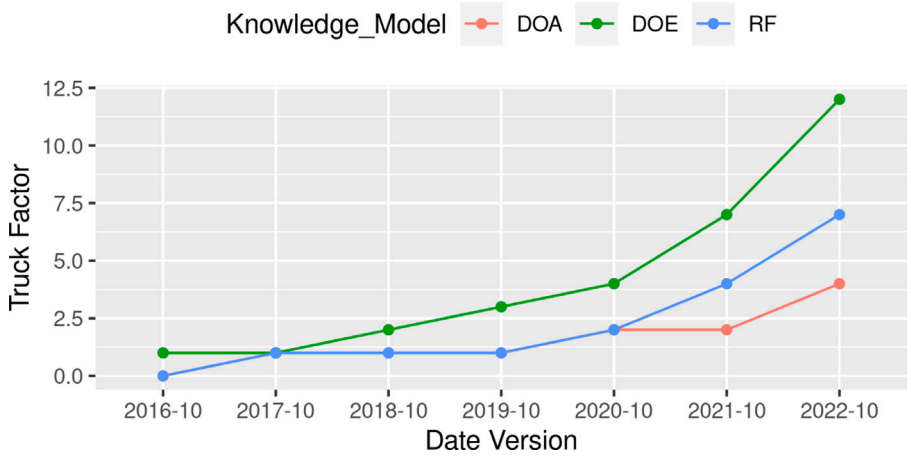


Fig. 8. Evolution of the Truck Factor according to the three metrics studied.

is also the possibility that only a few developers of a project responded to the survey. This may introduce a bias depending on the quality of the response and expertise of the respondents. However, since the responses are related to hundreds of files, this concern is somehow attenuated. In other words, our strategy tends to prioritize file coverage rather than the completeness of the responses per file.

Regarding the risk of misinterpretation of the knowledge scale (1 to 5) used in the first survey, we provided a guide to the score's meaning. However, the concept of *expert* is subjective, which remains a threat. Another threat is the definition of *expert* from the survey responses. Developers with knowledge above three were considered file experts. We claim this is a more conservative division of the knowledge scale, applied in a similar study [8]. Another risk is relying on developer comments to build ground truth in the Truck Factor application. To mitigate this risk, we only consider comments from active developers, bringing more reliability to the opinions, a similar approach used in a related study [5].

Internal Validity. Other factors may have an influence on the knowledge that a developer has in a source code file, which was not considered in this study [6,9]. However, we limit ourselves to identifying experts using information from *VCS*, which are tools widely used, making the evaluated techniques applicable to most projects. Nevertheless, other variables can be extracted from the information contained in code repositories, which can also play a role in the process of identifying file experts. Among them, we can mention the iteration with files in the same module and the complexity of the changed code. However, we perceive these variables as more complex, more difficult to compute, and dependent on the programming language. For this reason, we focused on variables that are less dependent on the particularities of the software under analysis.

The knowledge models used in this paper are ultimately based on source code modifications made by project contributors and are sensitive to loss of history in migrations to code hosting platforms. In the selection of repositories in the first study, we mitigated this threat using a heuristic to detect systems whose part of the development was performed outside the platform and migrated to it, losing part of the development history. Avelino reports that a similar procedure was used in the selection of projects in his study [5], the same projects used in the Truck Factor study.

Conclusion Validity. As for the statistical tests, Spearman's *rho* is used to analyze the correlation between the extracted variables. This coefficient was chosen because it does not require a normal distribution or a linear relationship between the variables. There is no consensus on the interpretation of the correlation values returned by the test, however, we relied on interpretations that are also used in studies from different areas [52]. We use 10-fold cross-validation to assess the performance of the machine learning models, and we emphasize that the results may vary to some degree according to the number of folds chosen. Another threat to the conclusion is the selection bias in this cross-validation. There is the possibility of concentrating evaluations from the same developer on training folds, for example, and thus introducing a bias. To deal with this problem, we use 10-fold cross-validations with three repetitions, causing performance to be estimated with several different divisions of training and testing, seeking to reduce the possibility of bias.

External Validity. We tried to maximize the ability to generalize the results. For open-source systems, we used six popular programming languages in the two parts of this paper. Therefore, by not limiting the projects to a single programming language, we intended to reduce the impact that certain languages may have on the developers' familiarity assessment, and consequently in our results. In total, data from 113 systems are used for the knowledge models study, and 131 for the Truck Factor application. The small amount of data from private projects makes it difficult to generalize the results for this type of project, but these results can be generalized to projects with characteristics similar to those analyzed.

6. Conclusion

In this study, we investigated the use of knowledge models for the identification of source code file expertise based on information in Version Control Systems. From a dataset composed of public and industrial projects, we identified the variables that are most related to code knowledge. *First Authorship* and *Number of Lines Added* shown highest positive correlations with knowledge, while *Recency of Modification* and *File Size* have the highest negative correlation. Using variables from this correlation study, a proposed linear model named *Degree of Expertise* and *Machine Learning* models were evaluated for the identification of experts, with the performance of these models compared with other techniques. As a result, three *Machine Learning* algorithms (*Random Forest*, *SVM*, and *Gradient Boosting Machines*) achieved the best *F-Score* and *Precision*.

We also propose a practical application of the evaluated models using them to compute the Truck Factor of public and private systems. We showed that using our models, Avelino's Truck Factor Algorithm was able to include relevant missing developers of open-source projects with a best average *F-Score* of 74%, and it was perceived as more accurate by six development leaders in computing the Truck Factor of a private repository.

Practitioners can significantly benefit from adopting the models proposed in this work, which excel in identifying source code experts and have an application in computing a project's Truck Factor, mitigating

risks associated with developer loss. Given the widespread recognition of Avelino's Algorithm in Truck Factor computation, our models can seamlessly integrate into software development workflows. Variations of Avelino's algorithm have been incorporated into code quality assurance tools, exemplified by CodeScene,[21] underscoring the relevance and acceptance of our models. With a demonstrated improvement in Avelino's algorithm performance, our models prove practical in diverse scenarios, including open-source and industrial systems. Utilizing these advancements aids practitioners in efficiently identifying and managing key developers and critical modules, contributing to software project sustainability and success.

As a final note, we provide a replication package with the results of our analysis as well as the survey's answers, repository data, and scripts used in the paper. The replication package of the two studies is available at https://doi.org/10.5281/zenodo.10806775. To extract the data, we developed a Java application that extracts the development history with the help of scripts. The tool is hosted on GitHub,[22] and we are developing a web interface for future availability as an online service for analyzing the concentration of knowledge in open-source projects.

CRediT authorship contribution statement

Otávio Cury: Writing – original draft, Validation, Software, Methodology, Investigation, Conceptualization. **Guilherme Avelino:** Writing – original draft, Validation, Supervision, Methodology, Conceptualization. **Pedro Santos Neto:** Writing – original draft, Validation, Supervision, Methodology, Conceptualization. **Marco Túlio Valente:** Writing – original draft, Methodology, Investigation, Conceptualization. **Ricardo Britto:** Writing – original draft, Methodology, Investigation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Available zenodo link in the paper.

Acknowledgments

We would like to thank the developers (Ericsson, Maida Health, and Open Source projects) who took their time to answer the survey and provided essential information to our investigation. Additionally, we also thank CNPq (grant 403512/2021-2) for supporting this work.

References

[1] V. Rajlich, Software evolution and maintenance, in: Proceedings of the on Future of Software Engineering, 2014, pp. 133–144.

[2] P. Ralph, S. Baltes, G. Adisaputri, R. Torkar, V. Kovalenko, M. Kalinowski, N. Novielli, S. Yoo, X. Devroey, X. Tan, et al., Pandemic programming: how COVID-19 affects software developers and how their organizations can help (2020), 2005, arXiv preprint arXiv:2005.01127.

[3] H. Kagdi, M. Hammad, J.I. Maletic, Who can help me with this source code change? in: 2008 IEEE International Conference on Software Maintenance, IEEE, 2008, pp. 157–166.

[4] M. Ferreira, M.T. Valente, K. Ferreira, A comparison of three algorithms for computing truck factors, in: 2017 IEEE/ACM 25th International Conference on Program Comprehension, ICPC, IEEE, 2017, pp. 207–217.

[5] G. Avelino, L. Passos, A. Hora, M.T. Valente, A novel approach for estimating truck factors, in: 2016 IEEE 24th International Conference on Program Comprehension, ICPC, IEEE, 2016, pp. 1–10.

[6] T. Fritz, G.C. Murphy, E. Murphy-Hill, J. Ou, E. Hill, Degree-of-knowledge: Modeling a developer's knowledge of code, ACM Trans. Softw. Eng. Methodol. 23 (2) (2014) 1–42.

[7] J.R. da Silva, E. Clua, L. Murta, A. Sarma, Niche vs. breadth: Calculating expertise over time through a fine-grained analysis, in: 2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering, SANER, IEEE, 2015, pp. 409–418.

[8] G. Avelino, L. Passos, F. Petrillo, M.T. Valente, Who can maintain this code?: Assessing the effectiveness of repository-mining techniques for identifying software maintainers, IEEE Softw. 36 (6) (2018) 34–42.

[9] E. Jabrayilzade, M. Evtikhiev, E. Tüzün, V. Kovalenko, Bus factor in practice, 2022, arXiv preprint arXiv:2202.01523.

[10] V. Cosentino, J.L.C. Izquierdo, J. Cabot, Assessing the bus factor of git repositories, in: 2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering, SANER, IEEE, 2015, pp. 499–503.

[11] E. Constantinou, G.M. Kapitsaki, Identifying developers' expertise in social coding platforms, in: 2016 42th Euromicro Conference on Software Engineering and Advanced Applications, SEAA, IEEE, 2016, pp. 63–67.

[12] N. Almarimi, A. Ouni, M. Chouchen, M.W. Mkaouer, Csdetector: an open source tool for community smells detection, in: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2021, pp. 1560–1564.

[13] G. Avelino, E. Constantinou, M.T. Valente, A. Serebrenik, On the abandonment and survival of open source projects: An empirical investigation, in: 2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, ESEM, IEEE, 2019, pp. 1–12.

[14] F. Calefato, M.A. Gerosa, G. Iaffaldano, F. Lanubile, I. Steinmacher, Will you come back to contribute? Investigating the inactivity of OSS core developers in GitHub, Empir. Softw. Eng. 27 (3) (2022) 1–41.

[15] E.D. Canedo, R. Bonifácio, M.V. Okimoto, A. Serebrenik, G. Pinto, E. Monteiro, Work practices and perceptions from women core developers in oss communities, in: Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, ESEM, 2020, pp. 1–11.

[16] O. Cury, G. Avelino, P. Santos Neto, R. Britto, M. Túlio Valente, Identifying source code file experts, in: Proceedings of the 16th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, 2022, pp. 125–136.

[17] M.K. Hossen, H. Kagdi, D. Poshyvanyk, Amalgamating source code authors, maintainers, and change proneness to triage change requests, in: Proceedings of the 22nd International Conference on Program Comprehension, 2014, pp. 130–141.

[18] L. Hattori, M. Lanza, Syde: a tool for collaborative software development, in: Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering, Vol. 2, 2010, pp. 235–238.

[19] C. Bird, N. Nagappan, B. Murphy, H. Gall, P. Devanbu, Don't touch my code! examining the effects of ownership on software quality, in: Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering, 2011, pp. 4–14.

[20] G. Canfora, M. Di Penta, R. Oliveto, S. Panichella, Who is going to mentor newcomers in open source projects? in: Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering, 2012, pp. 1–11.

[21] E. Sülün, E. Tüzün, U. Doğrusöz, Reviewer recommendation using software artifact traceability graphs, in: Proceedings of the Fifteenth International Conference on Predictive Models and Data Analytics in Software Engineering, 2019, pp. 66–75.

[22] J. Krüger, J. Wiemann, W. Fenske, G. Saake, T. Leich, Do you remember this source code? in: 2018 IEEE/ACM 40th International Conference on Software Engineering, ICSE, IEEE, 2018, pp. 764–775.

[23] E.M. Lucas, T.C. Oliveira, D. Schneider, P.S.C. Alencar, Knowledge-oriented models based on developer-artifact and developer-developer interactions, IEEE Access 8 (2020) 218702–218719, http://dx.doi.org/10.1109/ACCESS.2020.3042429.

[24] H. Kagdi, M. Gethers, D. Poshyvanyk, M. Hammad, Assigning change requests to software developers, J. Softw. Evol. Process 24 (1) (2012) 3–33.

[25] H. Kagdi, D. Poshyvanyk, Who can help me with this change request? in: 2009 IEEE 17th International Conference on Program Comprehension, IEEE, 2009, pp. 273–277.

[26] E. Sülün, E. Tüzün, U. Doğrusöz, RSTrace+: Reviewer suggestion using software artifact traceability graphs, Inf. Softw. Technol. 130 (2021) 106455.

[27] S. Asthana, R. Kumar, R. Bhagwan, C. Bird, C. Bansal, C. Maddila, S. Mehta, B. Ashok, WhoDo: automating reviewer suggestions at scale, in: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2019, pp. 937–945.

[28] M. Chouchen, A. Ouni, M.W. Mkaouer, R.G. Kula, K. Inoue, WhoReview: A multi-objective search-based approach for code reviewers recommendation in modern code review, Appl. Soft Comput. 100 (2021) 106908.

[21] https://codescene.com/

[22] Publicly available at https://github.com/OtavioCury/knowledge-islands.

[29] J.E. Montandon, L.L. Silva, M.T. Valente, Identifying experts in software libraries and frameworks among GitHub users, in: 2019 IEEE/ACM 16th International Conference on Mining Software Repositories, MSR, IEEE, 2019, pp. 276–287.
[30] A. Sajedi-Badashian, E. Stroulia, Guidelines for evaluating bug-assignment research, J. Softw. Evol. Process (2020) e2250.
[31] C. Hannebauer, M. Patalas, S. Stünkel, V. Gruhn, Automatically recommending code reviewers based on their expertise: An empirical comparison, in: Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering, 2016, pp. 99–110.
[32] J. Anvik, G.C. Murphy, Determining implementation expertise from bug reports, in: Fourth International Workshop on Mining Software Repositories, MSR'07: ICSE Workshops 2007, IEEE, 2007, p. 2.
[33] P.C. Rigby, Y.C. Zhu, S.M. Donadelli, A. Mockus, Quantifying and mitigating turnover-induced knowledge loss: case studies of chrome and a project at avaya, in: 2016 IEEE/ACM 38th International Conference on Software Engineering, ICSE, IEEE, 2016, pp. 1006–1016.
[34] A. Karlsson, Driving development resilience: Analyzing truck factors across proprietary and open-source projects.
[35] G. Avelino, L. Passos, A. Hora, M.T. Valente, Assessing code authorship: The case of the Linux kernel, in: IFIP International Conference on Open Source Systems, Springer, Cham, 2017, pp. 151–163.
[36] G. Navarro, A guided tour to approximate string matching, ACM Comput. Surv. 33 (1) (2001) 31–88.
[37] I.M.S. Ibiapina, F.V.M. Alves, W.A.L. Lira, G.A. Silva, P.A.S. Neto, Inferência da Familiaridade de Código por Meio da Mineração de Repositórios de Software, Simp. Brasileiro Qual. Softw. SBQS (2017).
[38] G. Canfora, L. Cerulo, M. Di Penta, Identifying changed source code lines from version repositories, in: Fourth International Workshop on Mining Software Repositories, MSR'07: ICSE Workshops 2007, IEEE, 2007, p. 14.
[39] W.A.L. Lira, Um método para inferência da familiaridade de código em projetos de software (Master's thesis), Universidade Federal do Piauí, Teresina, 2016.
[40] F.V. de Moura Alves, P. de Alcântara dos Santos Neto, W.A.L. Lira, I.M. de Sousa Ibiapina, Analysis of code familiarity in module and functionality perspectives, in: Proceedings of the 17th Brazilian Symposium on Software Quality, 2018, pp. 41–50.
[41] R.-H. Pfeiffer, Identifying critical projects via pagerank and truck factor, in: 2021 IEEE/ACM 18th International Conference on Mining Software Repositories, MSR, IEEE, 2021, pp. 41–45.
[42] F. Rahman, P. Devanbu, Ownership, experience and defects: a fine-grained study of authorship, in: Proceedings of the 33rd International Conference on Software Engineering, 2011, pp. 491–500.
[43] J. Oliveira, M. Viggiato, E. Figueiredo, How well do you know this library? Mining experts from source code analysis, in: Proceedings of the XVIII Brazilian Symposium on Software Quality, 2019, pp. 49–58.
[44] X. Sun, H. Yang, X. Xia, B. Li, Enhancing developer recommendation with supplementary information via mining historical commits, J. Syst. Softw. 134 (2017) 355–368.
[45] F. Ferreira, L.L. Silva, M.T. Valente, Turnover in open-source projects: The case of core developers, in: Proceedings of the 34th Brazilian Symposium on Software Engineering, 2020, pp. 447–456.
[46] S. Akter, Recommending Expert Developers Using Usage and Implementation Expertise (Ph.D. thesis), University of Lethbridge, Canada, 2021.
[47] A. Liaw, M. Wiener, et al., Classification and regression by randomforest, R News 2 (3) (2002) 18–22.
[48] J. Weston, C. Watkins, Multi-Class Support Vector Machines, Tech. Rep., Citeseer, 1998.
[49] L.E. Peterson, K-nearest neighbor, Scholarpedia 4 (2) (2009) 1883.
[50] J.H. Friedman, Greedy function approximation: a gradient boosting machine, Ann. Statist. (2001) 1189–1232.
[51] D.W. Hosmer Jr., S. Lemeshow, R.X. Sturdivant, Applied Logistic Regression, vol. 398, John Wiley & Sons, 2013.
[52] B.R. Overholser, K.M. Sowinski, Biostatistics primer: part 2, Nutr. Clin. Pract. 23 (1) (2008) 76–84.
[53] P. Schober, C. Boer, L.A. Schwarte, Correlation coefficients: appropriate use and interpretation, Anesth. Analg. 126 (5) (2018) 1763–1768.
[54] C. Meek, B. Thiesson, D. Heckerman, The learning-curve sampling method applied to model-based clustering, J. Mach. Learn. Res. 2 (Feb) (2002) 397–418.
[55] V. Haratian, M. Evtikhiev, P. Derakhshanfar, E. Tüzün, V. Kovalenko, BFSig: Leveraging file significance in bus factor estimation, in: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2023, pp. 1926–1936.
[56] S.Y. Chyung, K. Roberts, I. Swanson, A. Hankinson, Evidence-based survey design: The use of a midpoint on the Likert scale, Perform. Improv. 56 (10) (2017) 15–23.
[57] J. Linaker, S.M. Sulaman, M. Höst, R.M. de Mello, Guidelines for Conducting Surveys in Software Engineering V. 1.1, Lund University, 2015.